Explanation: Phishing & Malicious Email Classifier Notebook

1. What This Notebook Does

The notebook builds a Deep Learning model that reads a message (SMS text, used here as a stand-in for an email) and predicts whether it is 'ham' (legitimate) or 'spam' (malicious/phishing). It uses the public SMS Spam Collection dataset and trains an LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) neural network to perform this binary classification.

2. Libraries Used

Library	Purpose in this notebook
kagglehub	Downloads the SMS Spam Collection dataset directly from Kaggle.
pandas	Loads and manipulates the dataset as a DataFrame (rename, drop, inspect columns).
string	Built-in module used to strip punctuation during text cleaning.
nltk	Provides the English stopword list and a word tokenizer for text preprocessing.
scikit-learn	TfidfVectorizer for feature extraction; train_test_split to split data; accuracy_score to evaluate results.
TensorFlow / Keras	Tokenizer and pad_sequences for turning text into padded integer sequences; Sequential, Embedding, LSTM, Dense and Dropout layers to build and train the neural network.
NumPy	Handles numerical arrays, e.g. converting predicted probabilities to 0/1 labels.

3. Step-by-Step Walkthrough

1. **Data Loading —** The dataset is downloaded with kagglehub and loaded into a pandas DataFrame using latin-1 encoding (needed because the file contains special characters). The notebook inspects the first rows and an extra column ('Unnamed: 2') to understand the raw structure.
2. **Cleaning the Columns —** Columns v1 and v2 are renamed to label and text. Three unused columns (Unnamed: 2/3/4, leftover blank fields from the source CSV) are dropped, leaving a clean two-column table.
3. **Label Encoding —** The text labels 'ham' and 'spam' are converted to numbers (0 and 1) since neural networks require numeric targets rather than text categories.
4. **Text Cleaning —** Every message is lower-cased and all punctuation is stripped using Python's string.punctuation list, so that words like 'Free!' and 'free' are treated the same.
5. **Stopword Removal —** Common low-meaning words (e.g. 'the', 'is', 'a') are removed using NLTK's English stopword list, so the model focuses on words that actually help distinguish spam from ham.

Tokenization and Padding (used for the LSTM model)

- **Tokenization:** Keras's Tokenizer scans the training text and assigns a unique integer to each of the most frequent words (max_words = 10,000). Each message is then converted from a string into a sequence of these integers via texts_to_sequences. Words not seen during training are mapped to a special '<unk>' (out-of-vocabulary) token.
- **Padding:** Messages naturally have different lengths, but a neural network needs fixed-size input. pad_sequences forces every sequence to the same length (max_len = 100) — shorter messages are padded with zeros at the end ('post' padding), and longer ones are cut off ('post' truncating).
- **Train/Test Split:** Before tokenizing, the data is split into 80% training and 20% testing sets using train_test_split, so the model's final accuracy is measured on unseen messages.

4. Building and Training the LSTM Model

- **Embedding layer:** Turns each integer word-ID into a dense 128-dimension vector, letting the model learn word meaning/relationships.
- **LSTM layer:** 128 units process the sequence of word vectors in order, capturing context and word dependencies across the message; dropout (0.2) and recurrent dropout (0.2) reduce overfitting.
- **Dense + Dropout layers:** A 64-unit ReLU layer learns higher-level patterns, followed by a 50% dropout layer for regularization.
- **Output layer:** A single neuron with a sigmoid activation outputs a probability between 0 and 1 (spam likelihood) for binary classification.
- **Compilation & Training:** The model is compiled with the Adam optimizer and binary cross-entropy loss, then trained for 10 epochs with batch size 32, holding out 10% of the training data for validation. The trained model is saved as lstm.keras.

5. Building and Training the GRU Model

- **Embedding layer:** Turns each integer word-ID into a dense **128-dimensional vector**, allowing the model to learn word meaning and relationships.
- **GRU layer:** A **128-unit GRU** processes the sequence of word vectors in order, capturing contextual information and dependencies across the message; **dropout (0.2)** and **recurrent dropout (0.2)** help reduce overfitting.
- **Dense + Dropout layers:** A **64-unit ReLU** layer learns higher-level patterns from the GRU output, followed by a **50% dropout** layer for regularization.
- **Output layer:** A single neuron with a **sigmoid activation** outputs a probability between 0 and 1, representing the likelihood that the message is spam for binary classification.
- **Compilation:** The model is compiled using the **Adam optimizer**, **binary cross-entropy loss**, and **accuracy** as the evaluation metric.

6. Evaluation

The trained model predicts probabilities for the test set (model.predict). Each probability is converted to a hard label using a 0.5 threshold — values of 0.5 or above become 1 (spam) and the rest become 0 (ham). Finally, scikit-learn's accuracy_score compares these predicted labels against the true test labels to report how accurately the model classifies unseen messages.

Both the LSTM and GRU models achieved same accuracy score (0.8654)

7. Why Tokenization and Padding Matter Here

Neural networks only understand numbers, not raw words, so every piece of text must be converted into a fixed-size numeric form before it can be fed to the Embedding and LSTM layers:

- **Tokenization turns words into IDs:** each unique word learned from the training data gets its own integer index (for example, 'free' might become 42). A message becomes a list of these integers in the same order the words appeared, and the Embedding layer later turns each ID into a learned vector of numbers.
- **Padding standardizes length:** since messages vary in word count, the LSTM needs every input to be exactly the same length (100 in this notebook). Short messages get zeros appended at the end; long ones are trimmed. Without padding, the messages could not be batched together and trained efficiently.
- **Out-of-vocabulary handling:** any word not among the top 10,000 most frequent training words is replaced with the '<unk>' token, so the model still has a valid number to process instead of crashing on unseen words at test time.

7. Notebook Structure at a Glance

Section	Stage	What Happens
1	**Data Loading**	Download SMS Spam Collection dataset via kagglehub; load into a pandas DataFrame.
2	**Preprocessing**	Rename/drop columns, encode labels as 0/1, lowercase & remove punctuation, remove stopwords.
2	**Tokenization (Keras)**	Keras Tokenizer maps words to integer IDs; pad_sequences fixes every message to length 100.
3	**Model Building**	Sequential model: Embedding -> LSTM -> Dense(ReLU) -> Dropout -> Dense(sigmoid).
4	**Training**	model.fit for 10 epochs, batch size 32, 10% validation split; model saved as lstm.keras.
5	**Evaluation**	Predict on test set, threshold at 0.5, compute accuracy_score against true labels.

8. Summary

In short, the notebook takes raw SMS/email text, cleans and simplifies it (lowercasing, punctuation and stopword removal), converts it into fixed-length numeric sequences (tokenization + padding), and feeds those sequences into an Embedding + LSTM deep learning model that learns to tell spam from legitimate messages. The final accuracy score shows how well this pipeline generalizes to messages the model has never seen before.